

computer science. At a casual glance, the two problems look very similar; however, that is not the case. Finding a Eulerian cycle is a problem that belongs to a group of problems in class P, all of which have computationally feasible solutions. Finding a Hamiltonian cycle is a problem that belongs to a group of problems in class NP-complete, and so far, none of these have computationally feasible solutions. Whether problems in class NP-complete have computationally feasible solutions is one single greatest open problem (one of the seven Millennium Prize Problems selected by the Clay Mathematics Institute) in computer science. Any person who solves this problem will receive a Turing Award, Abel Prize, Fields Medal (if the person is under the age of 40), along with US\$ 1 million from the Clay Mathematics Institute, and many more accolades.

So far, we have only generated undirected simple graphs (having at most one edge between a pair of vertices). Let us again consider the case of roads in a city. Suppose there are two parallel roads connecting two places in a city. How do we represent them in a graph? Here, multigraphs come to our help. A multigraph can have loops (edges starting and ending at the same vertex) and parallel edges (more than one edge between a pair of vertices). The program `'mgraph7.sagews'` shown below generates a multigraph called *Mgraph_7* (line numbers have been included for clarity in the explanation).

```
1. MG = Graph(multiedges=True,
2. loops=True)
3. MG.add_vertices([1, 2, 3, 4])
4. for i in [1, 2]:
5.     MG.add_edge(1, 2)
6.     MG.add_edge(2, 3)
7.     MG.add_edge(3, 4)
8.     MG.add_edge(4, 1)
9.     MG.add_edge(i+2, i+2)
10. MG.show()
```

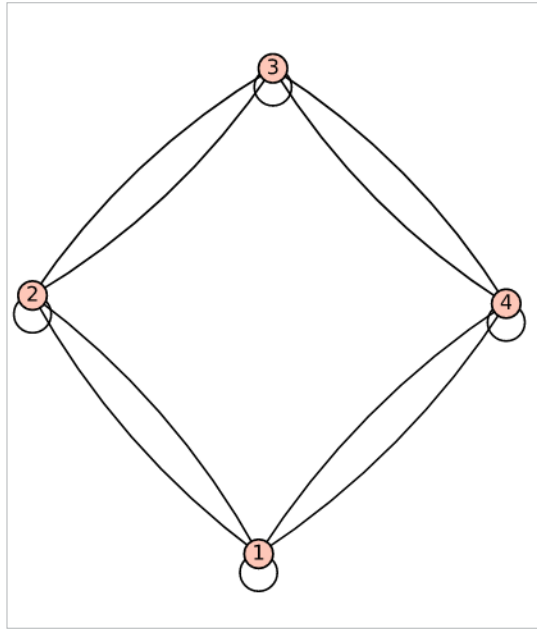


Figure 5: *Mgraph_7* generated by SageMath

Now, let us go through the code for better understanding. Line 1 creates a new graph object called *MG* with the `Graph()` constructor. The argument `multiedges` is set to `True`, indicating that the graph can have multiple edges between the same pair of vertices. The argument `loop` is also set to `True`, indicating that the graph can have loops. Line 2 adds vertices labelled 1, 2, 3, and 4 to the graph. Line 3 starts a `for` loop that iterates over the values 1 and 2. Lines 4 to 7 add edges between vertices 1 and 2, 2 and 3, 3 and 4, and 4 and 1. Since the `for` loop iterates twice, two parallel edges are created between these pairs of vertices. Lines 8 and 9 create loop edges. On the first iteration of the `for` loop, loop edges are added to vertices 1 and 3. On the second iteration of the `for` loop, loop edges are added to vertices 2 and 4. Finally, Line 10 displays the graph. It will show all the edges, including loops and multiple edges, as

configured in the graph creation step. Upon execution of the program `'mgraph7.sagews'`, we get the multigraph called *Mgraph_7* shown in Figure 5.

Now, let us list all the loops in the multigraph *Mgraph_7*. To do that, add the following line of code at the end of the program called `'mgraph7.sagews'`.

```
MG.loops(labels=False)
```

Upon execution of the modified version of the program `'mgraph7.sagews'`, we get the following output. From the output, we can see

that loops are created on vertices 1, 2, 3, and 4.

```
[(1, 1), (2, 2), (3, 3), (4, 4)]
```

Now, let us wrap up our discussion. In this third article in the series on SageMath, we explored various methods for creating undirected simple graphs. We also delved into techniques for extracting information from a graph when it represents a real-world problem in an abstract manner. Additionally, we examined the construction of multigraphs.

However, our exploration of graph theory in the context of SageMath is far from over. In the upcoming articles in this series, we will delve further into the topic of graphs. Why? Because graphs are an incredibly powerful data structure with applications in solving numerous real-world problems. **END** 🐼

By: Dr Deepu Benson

The author is a free software enthusiast, and his area of interest is theoretical computer science. He maintains a technical blog.